

LOCAL SEQUENCE ALIGNMENT OF DNA AND PROTEIN SEQUENCES USING SMITH–WATERMAN ALGORITHM

Project Review — Description, Implementation & Outputs

B. Jamuna — 2520030408

B. Pemesh — 2520030388

P. Anuradha Nandini — 2520030171

ABSTRACT

- Java-Based Bioinformatics Application
- Java application for comparing DNA and protein sequences.
- Implements the Smith–Waterman algorithm for local sequence alignment.
- Builds a 2D dynamic programming scoring matrix.
- Allows users to set match, mismatch, and gap scores.
- Uses traceback to find the best local alignment.
- Displays aligned sequences, alignment score, and percentage similarity.
- Helps identify conserved regions and mutations.
- Can be used to compare genes and protein sequences.
- Provides an easy way to visualize sequence similarities.
- Demonstrates the use of dynamic programming in bioinformatics.
- Helps analyze relationships and variations between biological sequences.

WHAT IT PRODUCES

- 1 Aligned sequences
- 2 Alignment score
- 3 Match / mismatch / gap counts
- 4 Percentage similarity

PROBLEM STATEMENT

Traditional sequence comparison methods struggle to find highly similar regions inside larger sequences once mutations, insertions, or deletions are present.

1

INPUT

Two DNA or protein sequences, entered or uploaded by the user.

2

PROCESS

Compute the optimal local alignment with configurable match / mismatch / gap scores.

3

OUTPUT

Best-matching regions, alignment score, and percentage similarity.

PROPOSED FUNCTIONALITIES

- Sequence input & validation (DNA / protein)
- Local alignment via Smith–Waterman
- Configurable match / mismatch / gap scores
- Alignment matrix generation & traceback
- Similarity calculation (score, matches, mismatches, gaps, %)
- DNA and protein support
- Clear, user-friendly result visualization
- Performance analysis across sequence lengths
- Core system built in Java with arrays & data structures
- Final alignment report generation

10+

functionalities

2

sequence types

3

configurable scores

2

algorithm modules (CO2 + CO3)

TOOLS & TECHNOLOGIES

Category	Tools / Technologies	Purpose
Programming Language	Java (JDK 21)	Core application + algorithms
Algorithms	Suffix Array + Kasai LCP, Smith–Waterman	CO2 exact matching → CO3 local alignment
Data Structures	2D Arrays, Strings, int[] arrays	Alignment matrix, suffix/LCP arrays
File Handling	Java I/O	Read/process sequences from text files
Version Control	Git & GitHub	Manage and maintain project source code

DESCRIPTION

A Two-Stage Pipeline Across Two Modules

CO2 — STRING ALGORITHMS

Suffix Array + Kasai LCP

Finds the longest EXACT common substring between the two sequences — a fast, exact conserved-region candidate. $O(n \log^2 n)$ build, $O(n)$ LCP.



CO3 — ADVANCED DP

Smith–Waterman

Finds the best-scoring LOCAL alignment, tolerating mismatches and gaps — how real mutations show up. $O(n \cdot m)$ DP + traceback.

Why in this order: a suffix array can only find exact matches, but is very fast. Smith–Waterman handles mismatches/gaps but costs $O(n \cdot m)$. Real aligners (e.g. BLAST) use this same “seed, then extend” pattern.

Both algorithms are implemented from first principles in plain Java — no external alignment libraries.

IMPLEMENTATION

Stage 1 (CO2) — Suffix Array + Kasai's LCP Algorithm

1 Build combined string

seqA + separator + seqB

2 Suffix Array (prefix doubling)

Sort all suffixes by rank pairs; $O(n \log^2 n)$

3 Kasai's LCP array

Longest common prefix between suffix-array neighbours; $O(n)$

4 Scan for cross-string LCP peaks

Best LCP between a suffix from A and a suffix from B = longest common substring

Used for: fast exact conserved-motif detection between the two input sequences, before scored alignment runs.

IMPLEMENTATION

Stage 2 (CO3) — Smith–Waterman Local Alignment

DP RECURRENCE

```
H(i,j) = max {  
  
    0  
    H(i-1,j-1) + s(ai,bj)    [match/mismatch]  
    H(i-1,j) - gap              [deletion]  
    H(i,j-1) - gap              [insertion]  
  
}
```

ALGORITHM STEPS

- Fill the (n+1)×(m+1) scoring matrix H, never letting a cell go below 0
- Track the highest-scoring cell in the matrix
- Traceback from that cell along the stored direction until a 0 is hit
- Reconstruct the aligned sequences + gap characters from the traceback path

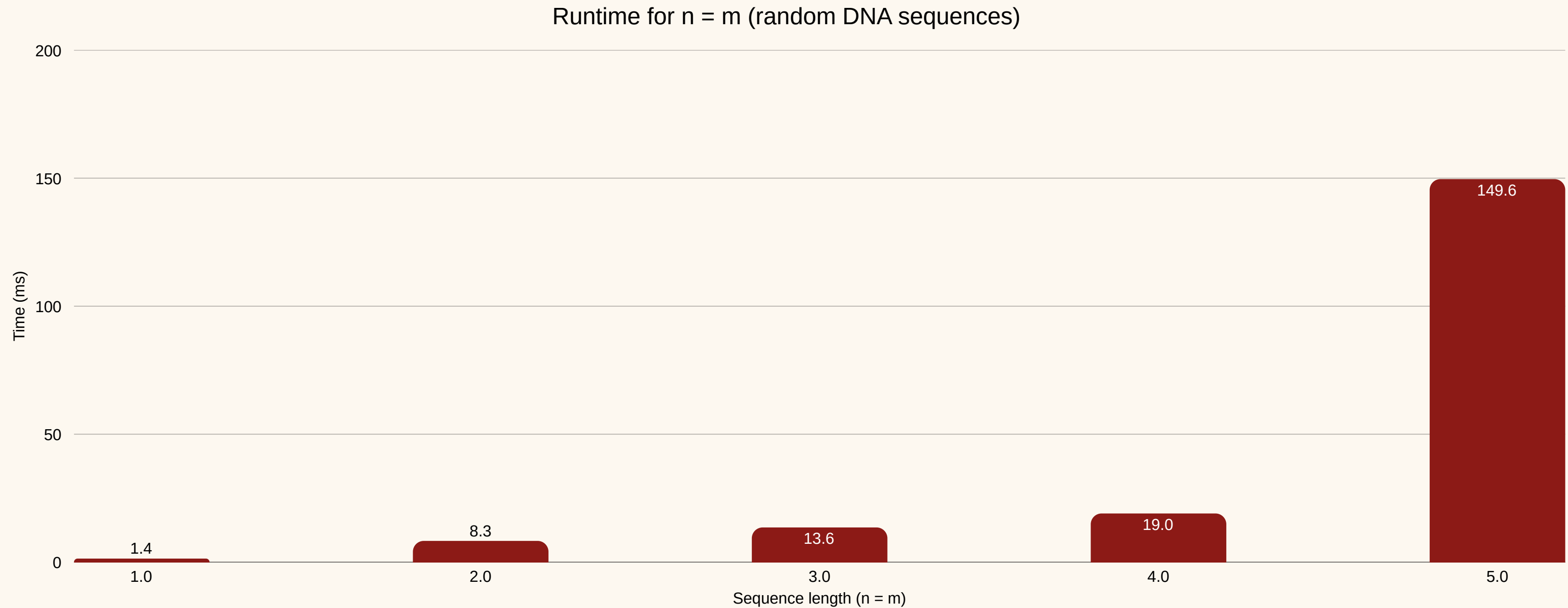
REAL RUN — DNA EXAMPLE (match +2, mismatch -1, gap -2)

A: GATTACA-GATTCAGGA
 ||| ||| ||||| ||
B: GATCACAGGATTCATGA

SCORE	SIMILARITY	MATCHES	GAPS
24	82.35%	14	1

OUTPUTS

Performance Analysis — Smith–Waterman Runtime vs. Sequence Length



Confirms the expected $O(n \cdot m)$ growth of the DP alignment stage — measured on the actual Java implementation, not estimated.

OUTPUTS

Protein Example + CO2 Exact-Match Stage

CO2 STAGE OUTPUT (both examples)

DNA pair → longest common substring “GATTCA” (length 6)

Protein pair → longest common substring “DL” (length 2)

Both found via the same Suffix Array + Kasai LCP routine — confirms the CO2 implementation generalizes across sequence type.

SEQUENCE VALIDATION

Every input is checked against the DNA alphabet (A,C,G,T) or the 20-letter amino-acid alphabet before either algorithm runs.

SMITH–WATERMAN OUTPUT (protein pair, match +1 / mismatch -1 / gap -2)

A: DP
 ||
B: DP

SCORE

2

SIMILARITY

100%

MATCHES

2

MISMATCHES

0

Short, exact overlap here — shows the algorithm correctly reports a small local match rather than forcing a long, low-quality alignment.

EXPECTED OUTCOMES

- Accurately compares DNA and protein sequences using Smith–Waterman
- Identifies the most similar local regions and their alignment score / % similarity
- Reports matches, mismatches, insertions, and deletions
- Configurable scoring parameters for different use cases
- A fast CO2 pre-filter (suffix array + LCP) alongside the CO3 scored alignment
- A useful tool for conserved-region identification and mutation analysis

THANK YOU